Looping Through Lists

Read, total, filter, transform and nest, safely

Loop over the items

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit.upper())
```

OUTPUT

```
APPLE
BANANA
CHERRY
```

Total and average

```
nums = [4, 8, 15, 16]
total = 0
for n in nums:
    total += n
print(total, total / len(nums))
```

OUTPUT

```
43 10.75
```

List comprehension shortcut

`[expression for item in list if condition]`

```
[n for n in nums if n % 2 == 0]  # → [4, 8, 16]
[n * n for n in nums]  # → [16, 64, 225, 256]
[f[0] for f in fruits]  # → ['a', 'b', 'c']
```

Unpack tuples in the header

```
pairs = [("Asha", 88), ("Ravi", 72)]
for name, mark in pairs:
    print(f"{name}: {mark}")
```

OUTPUT

```
Asha: 88
Ravi: 72
```

Never remove items while looping

```
a = [1, 3, 5, 2]
for x in a:
    if x % 2 == 1:
        a.remove(x)      # skips neighbours!
print(a)                 # → [3, 2]
a = [1, 3, 5, 2]
a = [x for x in a if x % 2 == 0]
print(a)                 # → [2]
```

WATCH OUT Removing shifts later items left, so the loop jumps over one. Build a new list, or loop over a copy: `for x in a[:]`.

Loop by index (use sparingly)

```
for i in range(len(fruits)):
    print(i, fruits[i])
```

OUTPUT

```
0 apple
1 banana
2 cherry
```

TIP Need the position as well? `enumerate` is cleaner (see sheet 06).

Filter into a new list

```
evens = []
for n in nums:
    if n % 2 == 0:
        evens.append(n)
print(evens)             # → [4, 8, 16]
```

Part of a list, reversed, sorted

```
for n in nums[:2]: ...           # first two
for n in nums[::-1]: ...         # reversed copy
for n in reversed(nums): ...     # no copy made
for n in sorted(nums, reverse=True): ...
```

Lists of lists

```
grid = [[1, 2], [3, 4], [5, 6]]
for row in grid:
    print(row, sum(row))
flat = [v for row in grid for v in row]
print(flat)
```

OUTPUT

```
[1, 2] 3
[3, 4] 7
[5, 6] 11
[1, 2, 3, 4, 5, 6]
```

Handy built-ins for lists

Call	Gives you
`len(x)`	how many items
`sum(x)` `min(x)` `max(x)`	total, smallest, largest
`sorted(x)`	a new sorted list
`any(x)` `all(x)`	is some / every item truthy
`v in x`	membership test
`x.count(v)` `x.index(v)`	how often / first position

REMEMBER

- Loop over the items directly; reach for an index only when you truly need it.
- `append` inside a loop builds a new list; a comprehension does the same in one line.
- Never add or remove items from the list you are looping over.
- Unpack tuples in the loop header: `for name, mark in pairs`.
- An empty list simply skips the loop. No special case needed.
- Use `sum`, `min`, `max`, `any`, `all` instead of hand-written loops when they fit.